Caching Strategy - Cheat Sheet

Caching is the engineering shortcut. Compute or fetch something expensive once, keep the result, serve it for free until it goes stale. Done right, caches transform performance and cost. Done wrong, they introduce stale data, race conditions, and consistency bugs that take months to find.

The hard parts are deciding when to invalidate, where to put the cache, and how to handle the moment everyone misses at once. The cache itself is easy.

Why Caching

Three things caches buy you:

- **Latency.** Serving a cached value beats hitting the DB or an upstream service every time.
- **Throughput.** Fewer DB calls means the DB can serve more unique requests.
- **Cost.** Fewer calls to paid APIs, fewer compute cycles spent on the same work.

The famous Phil Karlton quote: “There are only two hard things in Computer Science: cache invalidation and naming things.” The first half is no joke.

Where to Cache (Layers)

Caches live everywhere. Each layer solves a different problem.

Layer	What it caches	Examples
Browser	Static assets, API responses	HTTP Cache-Control, Service Worker
CDN	Static assets, edge-cached APIs	CloudFront, Cloudflare, Fastly
Reverse proxy	HTTP responses near the app	Varnish, nginx cache, Envoy
Application (in-process)	Hot objects in the JVM/process	Caffeine, Guava Cache, Ehcache local
Distributed cache	Shared across instances	Redis, Memcached, Hazelcast
Database	Query results, materialized views	Postgres query cache, MySQL buffer pool
Hardware	CPU caches, page cache	L1/L2/L3, OS page cache

Caches closer to the user are faster but harder to invalidate. Caches closer to the data are slower but easier to keep correct.

Cache Access Patterns

How your code reads from and writes to the cache.

1. Cache-aside (lazy loading)

The application checks the cache. On miss, it fetches from the source and populates the cache.

```
public User getUser(long id) {
    User user = cache.get(id);
    if (user == null) {
        user = db.findUser(id);
        if (user != null) {
            cache.put(id, user);
        }
    }
    return user;
}
```

Pros: simple, cache only holds what you actually use, source of truth stays in the DB.

Cons: every cache miss hits the DB. Initial requests are slow.

2. Read-through

The cache itself knows how to load missing data. The application just asks the cache.

```
LoadingCache<Long, User> cache = Caffeine.newBuilder()
    .maximumSize(10_000)
    .expireAfterWrite(Duration.ofMinutes(10))
    .build(db::findUser);

User user = cache.get(id);
```

Pros: cleaner application code, single place for loading logic.

Cons: cache library coupling, partial failures are harder to handle.

3. Write-through

Writes go to the cache, which synchronously writes to the source.

```
public void updateUser(User user) {
    cache.put(user.id(), user);
    db.save(user); // happens inside the cache wrapper
}
```

Pros: cache always in sync with the DB.

Cons: every write pays the DB latency. A cache write that fails after the DB write succeeds leaves inconsistency.

4. Write-behind (write-back)

Writes go to the cache. The cache asynchronously flushes to the source.

Pros: very fast writes from the caller's perspective.

Cons: data loss risk if the cache dies before flushing. Stronger consistency requirements need careful handling.

5. Refresh-ahead

The cache proactively refreshes entries before they expire, so users never see a slow miss.

Caffeine supports this via `refreshAfterWrite`. Useful for hot keys with predictable access.

```
LoadingCache<Long, User> cache = Caffeine.newBuilder()
    .maximumSize(10_000)
    .refreshAfterWrite(Duration.ofMinutes(5))
    .expireAfterWrite(Duration.ofMinutes(15))
    .build(db::findUser);
```

After 5 minutes, the next access triggers an async refresh while still returning the current value. After 15 minutes, the entry is gone.

Cache Update / Invalidation

The hard problem. Several common approaches.

TTL (time-to-live)

Each entry expires after a fixed time. Simple, but data is stale until the TTL is up.

```
cache.put(key, value, Duration.ofMinutes(5));
```

Best for data that tolerates some staleness (product catalog, user profile, configuration).

Explicit invalidation on write

When data changes, evict the cache entry.

```
public void updateUser(User user) {
    db.save(user);
    cache.evict(user.id());
}
```

Cleaner consistency. But “when data changes” can be hard to track across services. Easy to miss an evict path.

Event-driven invalidation

The source emits change events. Caches subscribe and invalidate their entries.

```
DB write -> outbox event -> message broker -> cache subscribers evict
```

Scales across services and instances. Adds infrastructure complexity (broker, subscription, ordering).

Versioned keys

Include a version in the cache key. When data changes, bump the version. Old entries fade out via TTL.

```
cache key: "user:42:v3"
```

When v4 ships, every read uses the new key. Old keys age out naturally. Useful when invalidation timing is hard to coordinate.

Eviction Policies

When the cache fills up, something has to go. The eviction policy decides what.

Policy	What it evicts	Good for
LRU (Least Recently Used)	The entry that hasn't been touched in the longest time	General purpose, the common default
LFU (Least Frequently Used)	The entry with the fewest hits	Long-tail workloads with stable hot keys
FIFO	The oldest entry	Simple, rare in practice
TLRU (Time-aware LRU)	LRU with TTLs	When freshness matters more than recency
Window TinyLFU	Hybrid LRU + LFU	Caffeine's default. Near-optimal hit rate.
Random	Random entry	Surprisingly competitive, no bookkeeping cost

Caffeine and modern Redis (with `allkeys-lfu`) use sophisticated policies that beat plain LRU in most workloads.

TTL and Freshness

Picking a TTL is a tradeoff between hit rate and staleness.

- **Short TTL (seconds):** very fresh, low hit rate, more load on the source.
- **Medium TTL (minutes):** balanced for most data.
- **Long TTL (hours):** high hit rate, very stale. Pair with explicit invalidation.
- **No TTL:** entries live until evicted. Risky without other invalidation.

Add jitter to TTLs (about 10%) to avoid mass expiration. If 10,000 entries all expire at the same second, the next request for each one stampedes the source.

```
long ttlSeconds = baseTtl + ThreadLocalRandom.current().nextLong(baseTtl / 10);
```

Cache Consistency

A cache is a denormalized copy. The original can change. Pick a consistency model that fits the use case.

Model	Guarantee
Strong consistency	Every read sees the latest write. Expensive, rarely needed.
Read-your-writes	After a user writes, their next read sees the new value. Often achieved by bypassing the cache after a write.
Eventual consistency	All reads converge given enough time. The common default for caches.
Bounded staleness	Data is at most N seconds old. TTL gives you this.

In most systems, caching means accepting eventual consistency. Decide which user-visible flows can't tolerate it (payments, auth) and bypass the cache there.

Common Problems

Cache stampede / thundering herd

A popular cache entry expires. Hundreds of concurrent requests all miss. All of them hit the source at once. The source falls over.

Defenses:

- **Locking / single-flight.** Only one request rebuilds the entry. Others wait for the result.
- **Stale-while-revalidate.** Serve the stale value while one request refreshes it.
- **Probabilistic early refresh.** Each request has a small chance of refreshing before expiry. Spreads load.
- **Jittered TTLs.** Don't let all entries expire at once.

Caffeine's loading cache does single-flight automatically.

Hot key

One key gets disproportionate traffic and overwhelms the cache server holding it (in a distributed cache).

Defenses:

- **Replicate the hot key** across multiple cache nodes.
- **Local cache in front of the distributed cache** for the hottest keys (multi-level caching).
- **Add randomness to the key** (split the key into N variants).

Cache penetration

Requests for keys that never exist hit the cache, miss, hit the DB, miss, return nothing. Repeated queries for missing data hammer the DB.

Defenses:

- **Cache the “not found” result** with a short TTL.
- **Bloom filter** in front of the cache. Quickly tells you a key is definitely absent.

Cache avalanche

Many entries expire at once, or the cache restarts cold. Mass miss → mass DB hits → DB falls over.

Defenses:

- **Jittered TTLs.**
- **Warm the cache on startup** from a snapshot or by loading hot keys.
- **Stagger rolling restarts.**

Snowballing inconsistency

A cached value gets stale, downstream caches store the stale value, and downstream-downstream caches store theirs. Invalidating becomes an orchestration problem.

Defenses:

- **Short TTLs at deeper layers.**
- **Cache at one or two layers only** unless you really need more.
- **Use versioned keys** so old data is naturally bypassed.

Common Tools

Tool	Type	Notes
Caffeine	In-process (Java)	High-performance LRU/LFU. Modern default for JVM.
Guava Cache	In-process (Java)	Older, simpler. Caffeine is the modern replacement.
Ehcache	In-process or distributed	Long-standing, integrates with Hibernate.
Redis	Distributed	Versatile: cache, queue, pub/sub, data structures. Pick by default.
Memcached	Distributed	Simpler than Redis, multi-threaded, raw key-value.
Hazelcast	Distributed	JVM-native, supports compute and caching.
Varnish	HTTP cache	Reverse proxy with a powerful config language.
Cloudflare / CloudFront / Fastly	CDN	Edge caching for static assets and APIs.
Spring Cache	Abstraction (Java)	Annotation-based, works with multiple providers.

Redis is the workhorse for distributed caching. Caffeine is the workhorse inside the JVM.

Cache Sizing

A cache that holds everything is just storage. The size limit is what makes it interesting.

Sizing factors:

- **Working set:** the rows that get hit regularly. Aim to cover this comfortably.
- **Entry size:** average object size in memory.
- **Memory budget:** how much RAM you're willing to spend.
- **Hit rate target:** usually 80-95% on the hot path.

Measure the actual hit rate after deployment. If it's below 50%, your cache might be too small, your access pattern might be uniform, or your TTL is too short.

Caffeine example:

```
Cache<Long, User> cache = Caffeine.newBuilder()
    .maximumSize(100_000)              // bounded by count
    .expireAfterWrite(Duration.ofMinutes(10))
    .recordStats()                     // enable stats
    .build();

CacheStats stats = cache.stats();
double hitRate = stats.hitRate();
```

For Redis, monitor `evicted_keys` and `keyspace_misses`. If evictions are high, the cache is undersized for the working set.

Observability for Caches

You can't tune what you can't see. Measure:

- **Hit rate:** percentage of lookups that found a value. Below 50% means trouble.
- **Latency:** cache lookups should be sub-millisecond locally, low single-digit ms for Redis.
- **Eviction rate:** how often entries are pushed out.
- **Size:** current entry count and memory footprint.
- **Load time:** for read-through caches, how long the loader takes on a miss.
- **Refresh rate:** for refresh-ahead caches, how often async refreshes happen.

Trace cache misses with sampling. A small percentage of slow misses can dominate p99 latency for a service that otherwise looks fast.

Best Practices

- **Cache the result of the expensive computation.** That's where the savings come from. Caching a single DB row that wasn't slow to fetch buys you very little.
- **Pick a clear consistency model per use case.** Eventual consistency by default. Bypass the cache for strongly-consistent flows.
- **Bound the cache size.** Unbounded caches are memory leaks waiting to happen.
- **Use a real eviction policy.** Default LRU is fine. Caffeine's W-TinyLFU is better.
- **Add jitter to TTLs.** Avoid synchronized mass expiration.
- **Cache the "not found" case.** Especially when penetration could be exploited (user-supplied IDs).
- **Single-flight on misses.** Stop the stampede before it starts.
- **Measure hit rate.** A cache with a 20% hit rate is hurting more than helping.
- **Plan for cold starts.** Caches lose state on restart. Have a warm-up strategy.

- **Use the same cache key format everywhere.** Different formats means different cache misses for the same logical data.

Common Gotchas

- **Stale data with no expiration plan.** Caches without TTL or explicit invalidation become forever-stale silently.
- **Caching errors.** A 500 response cached for 5 minutes turns a transient failure into a sustained outage.
- **Caching personal data per user but using a shared key.** One user sees another's profile. Catastrophic. Include user ID in the key.
- **Caching mutable objects by reference.** Caller mutates the returned object, the cache now holds the mutated version. Cache immutable copies.
- **Caching too eagerly.** Caching things that change every request gives you misses without hits.
- **Caching things that are already fast.** Adding a cache in front of a sub-millisecond lookup adds latency for the miss case and complexity everywhere.
- **Network calls during cache load.** A loader that makes 5 RPC calls turns one cache miss into a 5-call latency spike. Use single-flight.
- **Forgetting to invalidate related entries.** Update user 42's address: invalidate `user:42`. But what about `address:42` and `orders-by-user:42`? Map the dependency graph.
- **Cache stampede on deploy.** Rolling restarts that drop the cache cause a flood of misses. Pre-warm hot keys.
- **Distributed cache as a single point of failure.** Redis goes down, every request bypasses the cache, the DB falls over. Have a degraded mode (local cache fallback, circuit breaker on cache calls).

Quick Reference

Concept	Key fact
Cache-aside	App checks cache, falls back to DB on miss
Read-through	Cache itself loads missing data
Write-through	Writes go to cache then synchronously to DB
Write-behind	Writes go to cache, async to DB
Refresh-ahead	Cache refreshes hot entries before expiry
TTL	Time-to-live for cache entries
LRU	Evicts least recently used (common default)
W-TinyLFU	Caffeine's policy, near-optimal hit rate
Cache stampede	Many concurrent misses on the same key
Hot key	One key absorbs disproportionate traffic
Cache penetration	Queries for missing keys bypass the cache
Single-flight	Only one request rebuilds an entry on miss
Eventual consistency	The common consistency model for caches
Jittered TTL	Avoid synchronized expiration
Caffeine	In-process JVM cache, modern default
Redis	Distributed cache, modern default